Multiplayer’a Giriş & Temel Kavramlar

---

1. Multiplayer Oyun Nedir? (Çok Oyunculu Oyun)

Multiplayer oyun, aynı anda birden fazla oyuncunun ortak bir oyun dünyasında etkileşimde bulunmasını sağlayan bir video oyunu türüdür. Bu etkileşim, iş birliği (co-op), rekabet (PvP - Player versus Player) veya her ikisinin bir karışımı şeklinde olabilir.

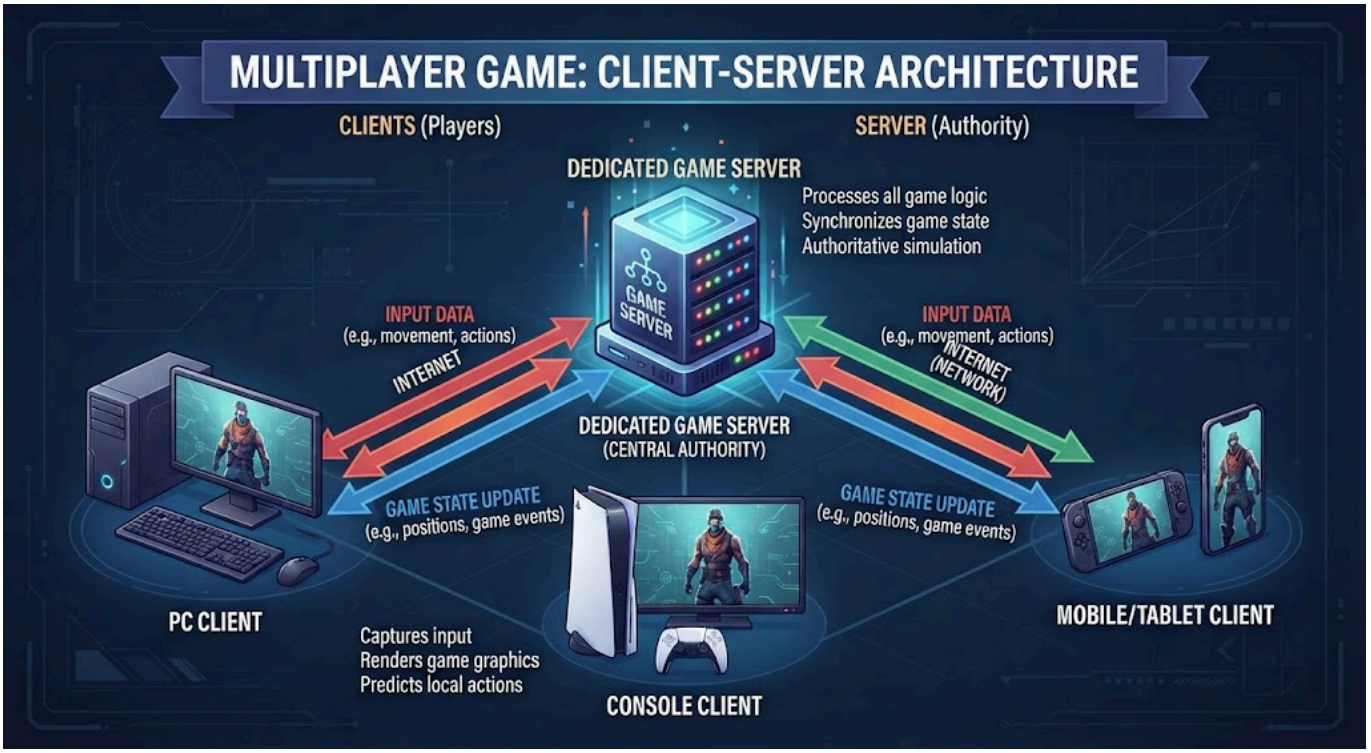
Multiplayer oyunların tarihsel gelişimi, aynı ekranı paylaşan (split-screen) yerel oyunlardan, günümüzde dünyanın dört bir yanındaki oyuncuları bir araya getiren devasa çevrimiçi (online) yapılara evrilmiştir. Bu oyunların temel amacı, sosyal bir deneyim sunmak ve yapay zeka (AI) kontrollü karakterlere kıyasla çok daha öngörülemez ve dinamik bir oyun ortamı yaratmaktır.

Multiplayer oyunları kabaca iki ana kategoriye ayırabiliriz:

- **Yerel (Local) Multiplayer:** Tüm oyuncular aynı fiziksel cihazda veya aynı yerel ağda (LAN) bulunur. Gecikme (latency) sorunu neredeyse yoktur.
- **Çevrimiçi (Online) Multiplayer:** Oyuncular internet üzerinden birbirine bağlanır. Bu yapı, daha karmaşık ağ mimarileri gerektirir ve performans, büyük ölçüde ağ koşullarına bağlıdır.

---

2. Client-Server Mantığı (İstemci-Sunucu Mimarisi)



Modern çevrimiçi multiplayer oyunların büyük çoğunluğu, Client-Server (İstemci-Sunucu) modeli üzerine inşa edilmiştir. Bu model, oyun dünyasının "doğru" ve "yetkili" (authoritative) bir kopyasını sunucuda tutar. İstemciler ise oyuncuların giriş yaptığı cihazlardır (bilgisayar, konsol, telefon).

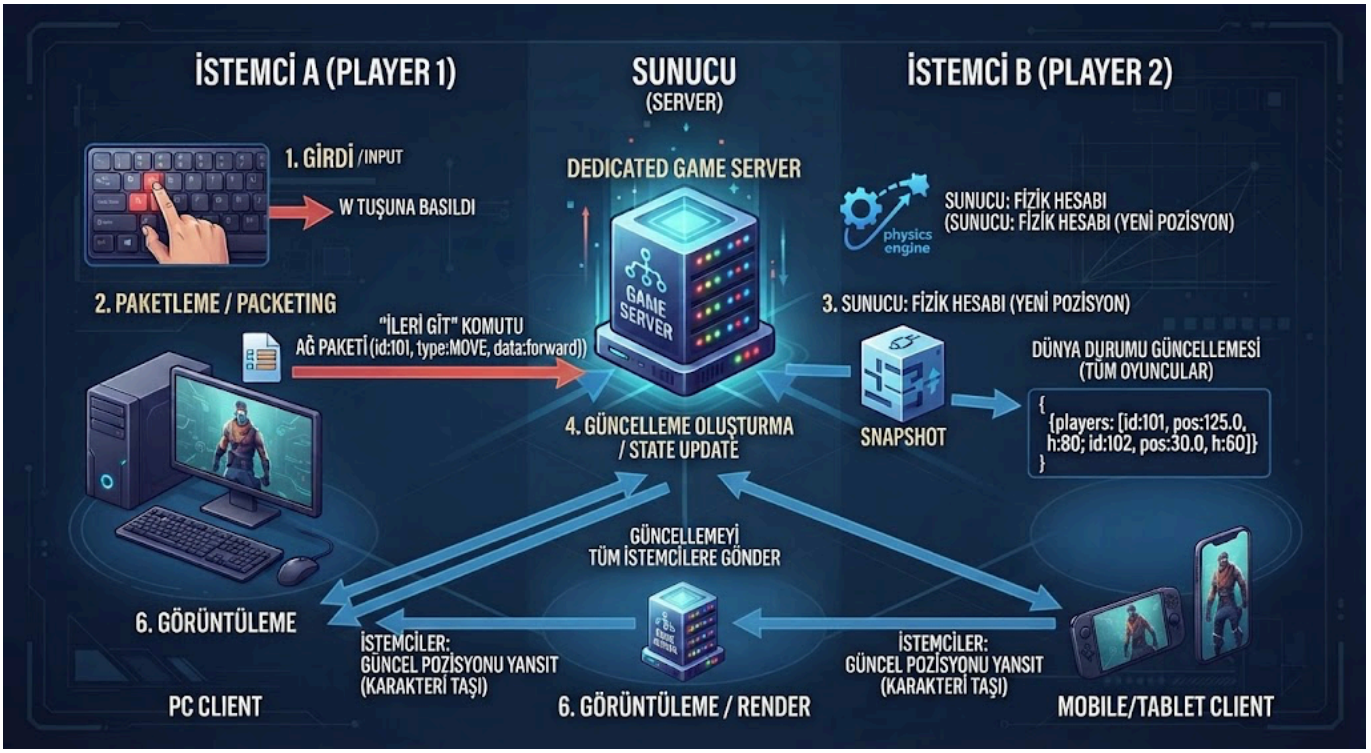
Temel mantığı şu şekildedir:

1. **Sunucu (Server):** Oyunun en üst otoritesidir. Oyun dünyasının tamamını hesaplar, tüm nesnelerin ve oyuncuların konumlarını, durumlarını, puanlarını ve diğer tüm verileri yönetir. Hile (cheat)

önlemenin temeli, sunucunun tüm önemli kararları almasıdır.
2. **İstemci (Client):** Oyuncunun oyunu oynadığı cihazdır. İstemci, oyuncunun klavye, fare veya gamepad gibi giriş cihazlarından aldığı komutları (input) paketleyerek sunucuya gönderir. Ayrıca, sunucudan gelen güncel dünya verilerini alır ve bu verileri kendi bilgisayarında görsel olarak render eder.

Örnek Bir Veri Akışı:

1. Oyuncu "W" tuşuna basar (İleri git komutu).
2. İstemci bu komutu "İleri git" emri olarak bir ağ paketine dönüştürür ve sunucuya iletir.
3. Sunucu, bu komutu alır, o oyuncunun karakterinin fizik kurallarına göre bir sonraki konumunu hesaplar.
4. Sunucu, tüm oyuncuların güncel konumlarını içeren bir "dünya durumu" güncellemesi (state update) oluşturur.
5. Sunucu, bu güncellemeyi bağlı olan tüm istemcilere gönderir.
6. İstemci, bu güncellemeyi alır ve ekrandaki karakteri yeni konumuna taşır.

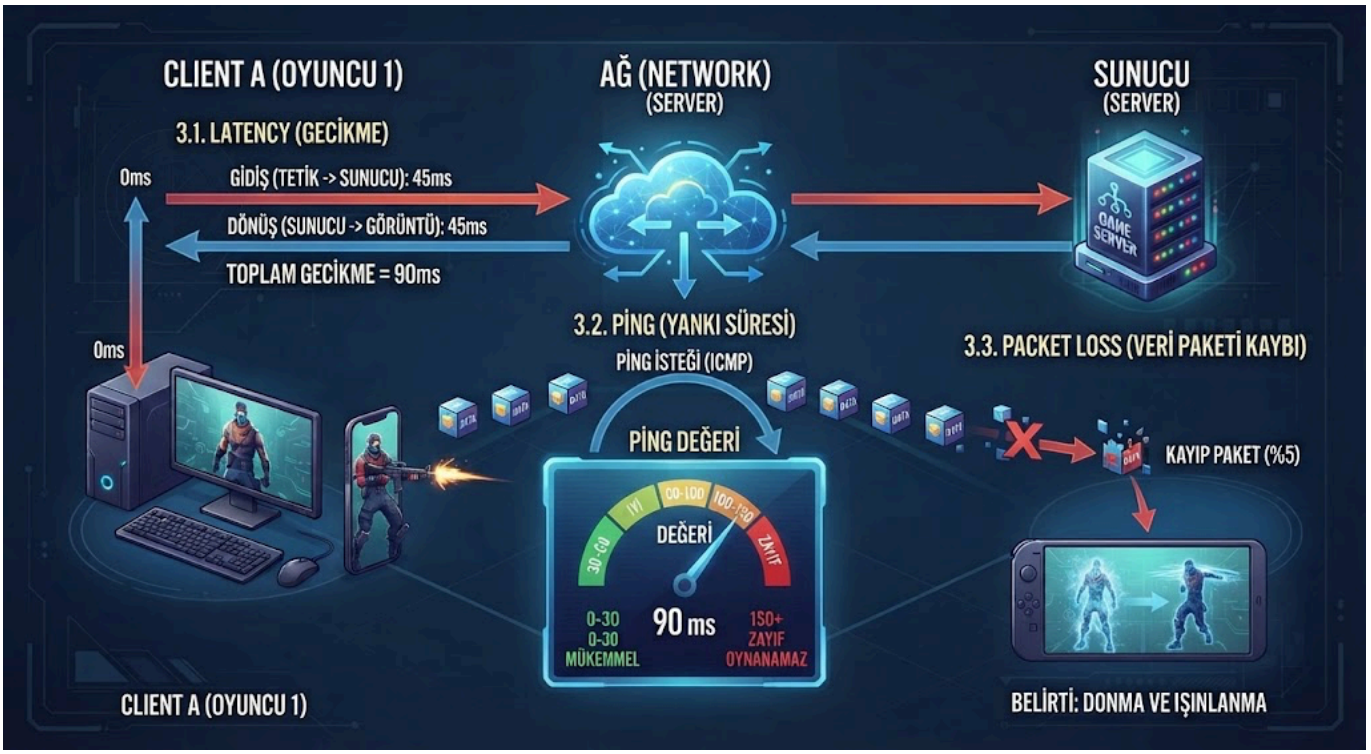


Bu mimarinin en büyük avantajları:

- **Yetki:** Hile yapmak çok zordur çünkü nihai karar sunucudadır. İstemci, "Ben 1000. seviyedeyim" diyemez.
- **Tutarlılık:** Tüm oyuncular, sunucunun onayladığı aynı gerçekliği görür.

---

3. Latency, Ping ve Packet Loss Kavramları



Çevrimiçi oyunlarda deneyim kalitesini belirleyen en kritik faktörler ağ performansıdır. Bu faktörler üç ana başlık altında toplanır:

3.1. Latency (Gecikme) Latency, bir veri paketinin kaynaktan (istemci) hedefe (sunucu) ulaşması için geçen zaman dilimidir. Genellikle milisaniye (ms) cinsinden ölçülür. Yüksek gecikme, oyuncunun yaptığı bir hareket ile bu hareketin sonucunun ekrana yansıması arasında gecikme olmasına neden olur. Bu durum, özellikle hızlı refleks gerektiren FPS (First-Person Shooter) gibi oyunlarda oyuncuyu dezavantajlı duruma sokar.

3.2. Ping (Yankı Süresi) Ping, genellikle Latency ile eş anlamlı olarak kullanılır ve genellikle bir ICMP (Internet Control Message Protocol) yankı isteğinin gönderilip geri gelmesi için geçen süreyi ifade eder. Oyunlarda, istemcinin sunucuya bir veri paketi gönderip, sunucunun aynı paketi geri göndermesi arasında geçen süre olarak ölçülür. Oyun lobilerinde gördüğümüz "Ping: 30 ms" ibaresi, bağlantı kalitesinin bir göstergesidir.

Ping Değeri (ms)	Oyun Deneyimi Açıklaması
0 - 30	Mükemmel. Gecikme neredeyse fark edilmez.
30 - 60	İyi. Rekabetçi oyunlar için kabul edilebilir.
60 - 100	Orta. Hafif bir his kayması olabilir, strateji oyunları için uygundur.
100 - 150	Zayıf. FPS gibi hızlı oyunlarda zorlayıcıdır.
150+	Oynanamaz. Ciddi gecikmeler ve geri sıçramalar yaşanır.

3.3. Packet Loss (Veri Paketi Kaybı) İstemci ve sunucu arasında gönderilen veri paketlerinin hedefine ulaşamaması durumudur. Bu paketler, ağdaki tıkanıklıklar, hatalı kablolar veya kablosuz sinyal zayıflığı nedeniyle kaybolabilir. Packet Loss oranı %1'den fazla ise oyunda ciddi aksaklıklar yaşanır. Belirtileri şunlardır:

- *Teleportasyon:* Karakterin aniden bir noktadan başka bir noktaya ışınlanması.
- *Hits not registering:* Yaptığınız atışların veya saldırıların hiçbir etki göstermemesi.
- *Donma ve Kopmalar:* Oyunun kısa süreliğine durması ve ardından hızla ilerlemesi.

Özet ve Çıkarımlar

- Multiplayer oyunlar, birden fazla oyuncuyu aynı sanal alanda buluşturan yapılardır.
- **Client-Server** modeli, **Sunucu**'nun otorite olduğu ve tüm hesaplamaları yaptığı, **İstemci**'nin ise sadece girdi gönderip sunucunun gönderdiği güncel durumu ekrana yansıttığı mimaridir.
- Oyun içi performans doğrudan ağ kalitesine bağlıdır.
- **Latency/Ping** düşük olmalıdır; ideal olarak 50 ms'nin altı tercih edilir. Ping, paketin gidiş-dönüş süresidir.
- **Packet Loss** sıfıra yakın olmalıdır. Bu değerin artması, oyunun bütünlüğünü bozar ve oynanabilirliği düşürür.

Tüm bu kavramlar, oyun geliştiricilerinin ağ kodlaması yaparken ve oyuncuların kendi oyun deneyimlerini optimize ederken göz önünde bulundurmaları gereken temel faktörlerdir.

Single Player vs Multiplayer Farklar

Tek oyunculu (Single Player) ve çok oyunculu (Multiplayer) oyunlar, oyuncuya sundukları deneyim, geliştirme süreçleri ve teknik altyapı gereksinimleri açısından köklü farklılıklar gösterir. Bu iki yapıyı karşılaştırmalı olarak inceleyelim.

---

1. Tanımsal Fark (Temel Ayrım)

Single Player	Multiplayer
Tek bir oyuncunun, yapay zeka (AI) tarafından kontrol edilen düşmanlar ve NPC'ler (Non-Player Character) ile etkileşime girdiği oyun türüdür. Hikaye ve ilerleme tamamen oyuncunun kendi hızına ve kararlarına bağlıdır.	Birden fazla insan oyuncunun aynı oyun ortamında buluştuğu, birbirleriyle iş birliği veya rekabet içinde olduğu oyun türüdür. Dinamikler, diğer insan oyuncuların kararlarına göre sürekli değişir.

---

2. Oynanış (Gameplay) ve Deneyim Farkları

- **Kontrol ve Otorite:**
    - *Single Player:* Oyuncu, oyun dünyasının mutlak merkezidir. Hikayenin akışı, görevlerin tamamlanma sırası ve zorluk derecesi büyük ölçüde oyuncunun tercihlerine göre şekillenir. Oyun, oyuncunun varlığına göre tasarlanmıştır.
    - *Multiplayer:* Otorite dağılmıştır. Diğer oyuncular da sizin gibi eşit (veya farklı) haklara sahiptir. Oyun dünyası sizin hareketlerinizden bağımsız olarak diğer oyuncuların eylemleriyle de evrilir. Öngörülemezlik, multiplayer deneyiminin temel taşıdır.
- **Zaman Algısı ve Duraklatma (Pause):**
    - *Single Player:* Oyun herhangi bir anda duraklatılabilir. Oyuncu, menüye girip strateji geliştirebilir, bir görevi yarıda bırakıp kapatabilir. Zaman, oyuncunun kontrolündedir.

    - *Multiplayer:* Oyun, diğer oyuncular devre dışı kalmadığı sürece devam eder. Duraklatma veya hızlandırma gibi müdahaleler genellikle mümkün değildir. Zaman, tüm oyuncular için eşit ve sürekli akar.

- **Hedef ve Motivasyon:**
    - *Single Player:* Hedef çoğunlukla hikayeyi tamamlamak, yan görevleri bitirmek, karakteri geliştirmek ve oyun dünyasını keşfetmektir. İlerleme, oyuncunun kişisel başarısıdır.
    - *Multiplayer:* Hedef, diğer insanlara karşı üstünlük sağlamak (rank atlama, liderlik tabloları), takım arkadaşlarıyla koordineli hareket ederek zorlu görevleri aşmak (raid, ekip oyunları) veya sosyal etkileşimin tadını çıkarmaktır. Motivasyon, dışa dönük ve karşılaştırmalıdır.

---

3. Teknik ve Geliştirme Farkları (Altyapı)

Kriter	Single Player	Multiplayer
Ağ Kodu (Netcode)	Gerekmez. Tüm hesaplamalar oyuncunun kendi cihazında (lokal) yapılır.	Zorunludur. Client-Server mimarisi, gecikme telafisi (lag compensation), veri senkronizasyonu gibi karmaşık sistemler gerektirir.
Performans Gereksinimi	Yalnızca oyuncunun donanımına bağlıdır. Daha düşük sistemlerde çalışacak şekilde optimize edilebilir.	Hem donanıma hem de internet bağlantısına (ping, paket kaybı) bağlıdır. Sunucu maliyetleri ve altyapı yatırımı gerektirir.
Veri Yönetimi	Kayıtlar (save dosyaları) oyuncunun cihazında veya bulutta kişisel olarak saklanır.	Tüm önemli veriler (oyuncu istatistikleri, envanter, başarımlar) merkezi sunucularda saklanır. Veritabanı yönetimi kritiktir.
Hata (Bug) Yönetimi	Bir hata bulunduğunda, oyuncu yamayı bekleyebilir veya mod ile düzeltebilir.	Hatalar çok daha kritiktir. Sömürü (exploit) oyun dengesini bozabilir. Hata tespiti ve düzeltme süreci acil ve süreklidir.
Hile (Cheat) Önleme	Hile yapmak yalnızca oyuncunun kendi deneyimini etkiler, bu nedenle önlemler minimaldir.	Hayati önem taşır. Anti-cheat yazılımları (Easy Anti-Cheat, BattleEye) ve sunucu tarafı doğrulamalar zorunludur. Hile, diğer oyuncuların deneyimini doğrudan mahveder.

---

4. İçerik ve Yaşam Döngüsü (Lifecycle) Farkları

- **Hikaye ve Anlatı:**
    - *Single Player:* Baştan sona planlanmış, başlangıcı, gelişimi ve sonu olan bir anlatı sunar. Yan görevlerle zenginleştirilir. *Örnek:* The Witcher 3, God of War.
    - *Multiplayer:* Merkezinde anlatıdan çok sistemler (rekabet, iş birliği, ekonomik döngü) vardır. Hikaye varsa bile arka planda kalır ve oyuncu tarafından oluşturulan anılarla (örneğin "Şu maçta nasıl geri döndük") şekillenir. *Örnek:* Valorant, World of Warcraft.

- **Tekrar Oynanabilirlik (Replayability):**

- *Single Player:* Genellikle senaryo bitiminde azalır. Farklı zorluk seviyeleri veya farklı karakter sınıfları ile tekrar oynanabilir, ancak bu sınırlıdır.
- *Multiplayer:* İnsan rakibin öngörülemezliği nedeniyle teorik olarak sonsuzdur. Her maç veya karşılaşma bir öncekinden farklıdır. İçerik güncellemeleri (sezonlar, battle pass) ile yaşam döngüsü yıllarca uzatılabilir.

---

5. Psikolojik ve Sosyal Farklar

- **Başarısızlık (Failure) Algısı:**
  - Single Player'da başarısızlık, oyuncunun kendi hatası veya zorluk seviyesi ile ilgilidir. Kaybeder, tekrar dener, öğrenir.
  - Multiplayer'da başarısızlık, genellikle başkaları tarafından gözlemlenir. Bu, kaygıya (rekabetçi oyunlarda "korku") veya takım arkadaşlarına yönelik hayal kırıklığına yol açabilir. Sosyal baskı mevcuttur.
- **Ödül ve Tatmin (Reward & Satisfaction):**
  - Single Player'da tatmin, hikayenin doruk noktasına ulaşmak, zorlu bir patronu yenmek veya oyunu %100 tamamlamaktan gelir. Ödül, bireyseldir.
  - Multiplayer'da tatmin, başka bir insanı zekanızla veya reflekslerinizle yenmekten veya takım olarak zafere ulaşmaktan gelir. Ödül, sosyal onay ve karşılaştırmalı üstünlüktür.

Özet Tablo

Özellik	Single Player	Multiplayer
Oyuncu Sayısı	1	2 veya daha fazla
Rakipler	Yapay Zeka (AI)	İnsan oyuncular
Duraklatma	Mümkün	Mümkün değil
İnternet İhtiyacı	Çoğunlukla gerekmez (DRM hariç)	Zorunlu
Temel Zorluk	Mekanik yetenek ve problem çözme	Koordinasyon, strateji ve refleks
Hile Etkisi	Kişisel deneyimi bozar	Tüm oyuncu kitlesinin deneyimini bozar
Yaşam Süresi	Senaryo ile sınırlı (30-100 saat)	Topluluk yaşadıkça sınırsız

---

Single player ve multiplayer oyunlar, aynı sektörün ürünleri olmalarına rağmen neredeyse birbirinden farklı iki sanat dalıdır. **Single player**, yönetmenin (geliştiricinin) kontrolünde, oyuncuya özel bir hikaye anlatıcılığıdır. **Multiplayer** ise geliştiricilerin yalnızca kuralları ve arenayı sağladığı, asıl içeriğin oyuncular tarafından üretildiği, dinamik ve sosyal bir sistemdir. Teknik altyapıdan oyun tasarımına kadar her karar, bu temel ayrıma göre şekillenir.

Basit "Message Exchange" Simülasyonu (Pseudo Network)##

Bu bölümde, istemci (client) ve sunucu (server) arasındaki veri alışverişini (message exchange) temel seviyede simüle eden bir mantık oluşturacağız. Gerçek ağ protokolleri (TCP/UDP) ve soket (socket) programlamaya girmeden önce, bu sürecin nasıl işlediğini anlamak için kurgusal (pseudo) bir model geliştireceğiz.

Amacımız, bir oyun içinde "Hareket Et" ve "Ateş Et" gibi iki temel komutun istemciden sunucuya nasıl gönderildiğini, sunucunun bunları nasıl işlediğini ve diğer istemcilere nasıl dağıttığını sırasıyla, adım adım göstermektir.

---

1. Sistem Mimarisi Varsayımları

Sistemimiz 3 ana bileşenden oluşur:

1. **İstemci A (Oyuncu 1):** Giriş gönderen taraf.
2. **İstemci B (Oyuncu 2):** Güncellemeleri alan taraf.
3. **Sunucu (Yetkili Otorite):** Veriyi doğrulayan, işleyen ve dağıtan merkez.

İletişim Kuralı: Tüm mesajlaşma, "Packet" (Veri Paketi) adı verilen yapılar üzerinden gerçekleşir. Her paket türü, içerdiği veriye göre farklı bir "Message Type" (Mesaj Türü) etiketine sahiptir.

---

2. Kullanılacak Veri Yapıları (Pseudo-Code)

Mesajlarımızı yapılandırılmış bir şekilde göndermek için bir "Packet" şablonu tanımlayalım. Bu, JSON (JavaScript Object Notation) formatına benzer şekilde düşünülebilir.

```
Packet Template:
{
  "senderId": (Gönderenin Benzersiz Kimlik Numarası),
  "type": (Mesaj Türü - Örn: "MOVE", "SHOOT", "STATE_UPDATE"),
  "timestamp": (Paketin oluşturulduğu an, ms cinsinden),
  "data": (İlgili türe göre değişen veri içeriği)
}
```

---

3. Adım Adım Simülasyon Senaryosu

Senaryo: İstemci A, karakterini sağa hareket ettiriyor ve ardından bir hedefe ateş ediyor. Sunucu bu hareketleri doğrulayıp, İstemci B'ye oyun dünyasının güncel halini gönderiyor.

Adım 1: İstemci A'dan Sunucuya - Hareket Mesajı (MOVE)

Oyuncu A, klavyesinde "Sağ Ok" tuşuna basar. İstemci A bu girişi (input) yakalar ve bir ağ paketine dönüştürür.

```
[Pseudo Paket - İstemci A Çıkışı]
{
  "senderId": 101,
```

```
  "type": "MOVE",
  "timestamp": 48500012,
  "data": {
    "direction": "right",
    "speed": 5.0
  }
}
```

Yorum: İstemci A, bu paketi UDP (User Datagram Protocol) gibi hızlı bir protokol ile sunucuya gönderir. "100 ms sonra" hedefe ulaşacağını varsayıyoruz.

Adım 2: Sunucu - Hareket Mesajını İşleme (Processing)

Sunucu, İstemci A'dan gelen paketi alır. Paketi işlemeden önce 3 temel kontrolden geçirir:

1. **Doğrulama (Validation):** Bu oyuncunun gerçekten sağa hareket etme hakkı var mı? (Örn: Harita sınırlarını kontrol eder). Burada geçerli kabul edelim.
2. **Kimlik Kontrolü:** Paketteki `senderId` (101) sunucu veritabanında kayıtlı aktif bir oyuncu mu? Kontrol başarılı.
3. **Zaman Damgası (Timestamp):** `timestamp` değeri, sunucunun mevcut zamanına uygun mu? (Çok eski bir paket mi?) Kabul edilebilir gecikme sınırları içinde.

Sunucu İç Mantığı (Pseudo-Code):

```
function onMoveReceived(packet):
    player = findPlayer(packet.senderId)
    if player is valid and player.canMove(packet.data.direction):
        // Oyunun fizik motorunu çalıştır
        player.x = player.x + (packet.data.speed * Time.deltaTime)
        player.lastMoveTime = packet.timestamp
        // İşlemin başarılı olduğunu belirten yeni durumu hafızada güncelle
        worldState[packet.senderId].position = player.x
        return SUCCESS
    else:
        return IGNORE // Geçersiz hareket, dikkate alınmadı
```

Adım 3: İstemci A'dan Sunucuya - Ateş Mesajı (SHOOT)

Hareketten kısa bir süre sonra İstemci A, "Fare Sol Tık" ile ateş eder. Bu da ayrı bir paket olarak gelir.

```
[Pseudo Paket – İstemci A Çıkışı]
{
  "senderId": 101,
  "type": "SHOOT",
  "timestamp": 48500550,
  "data": {
    "targetX": 150.0,
```

```
    "targetY": 200.0,
    "weaponId": 3
  }
}
```

Sunucu İşlemi: Sunucu bu paketi alır, hedef koordinatlarını kontrol eder (menzil içinde mi, hedef görüş hattında mı?), gerekli "isabet" (hit) hesaplamasını yapar ve eğer doğruysa, hedef oyuncunun (İstemci B) canını düşürür.

Adım 4: Sunucudan İstemci B'ye - Dünya Durum Güncellemesi (STATE_UPDATE)

Sunucu, İstemci A'nın hareketini ve ateşini başarıyla işledikten sonra, artık tüm bağlı istemcilere (özellikle bu olaylardan etkilenen İstemci B'ye) oyun dünyasının yeni "doğru" halini göndermek zorundadır.

Sunucu, tüm oyuncuların güncel konumlarını, can seviyelerini ve diğer dinamik nesneleri içeren yeni bir anlık görüntü (snapshot) oluşturur.

```
[Pseudo Paket - Sunucu Çıkışı (İstemci B'ye)]
{
  "senderId": 0,             // Sunucu
  "type": "STATE_UPDATE",
  "timestamp": 48500600,
  "data": {
    "players": [
      { "id": 101, "x": 125.0, "y": 50.0, "health": 80 },
      { "id": 102, "x": 30.0, "y": 75.0, "health": 60 }
    ],
    "bullets": [
      { "id": 55, "position": "150.0, 200.0", "status": "expired" }
    ],
    "serverTick": 123456
  }
}
```

Yorum: İstemci B, bu paketi alır. "Bullets" listesinde `status: "expired"` yazarak, İstemci A'nın ateş ettiği merminin hedefe ulaştığını ve artık görsel olarak render edilmemesi gerektiğini anlar. Ayrıca oyuncu 101'in konumunu güncelleyerek ekranı yeniden çizer.

---

4. Simülasyondaki Kritik Kavramlar

1. **Message Type (Mesaj Türü):** Paketlerin "MOVE", "SHOOT", "STATE_UPDATE" gibi türlere ayrılması, sunucunun aynı fonksiyonu çağırmak yerine doğru iş mantığına (logic) yönlendirilmesini sağlar. Bu ayrım, kodun okunabilirliği ve verimliliği için çok önemlidir.
2. **Timestamp (Zaman Damgası):** Simülasyonda her pakete bir zaman damgası ekledik. Bu, sunucunun gelen paketlerin sırasını (order) doğru tespit etmesi (Örn: Ateş etmeden önce hareket mi edildi?) ve aşırı gecikmeli paketleri reddetmesi için hayati öneme sahiptir.
3. **İstemciden Sunucuya (C2S) vs Sunucudan İstemciye (S2C):**

- **C2S (İstemci -> Sunucu):** Genellikle küçük, sık gönderilen ve hız gerektiren paketlerdir (Input komutları).
- **S2C (Sunucu -> İstemci):** Genellikle daha büyük, tüm oyuncuları kapsayan dünya durumu paketleridir (State Update). Sunucu burada bant genişliğini verimli kullanmak için veri optimizasyonuna gider.

---

5. NOt

Bu simülasyon, "Reliable UDP" veya "WebSocket" gibi gerçek protokollerle uygulanır. Oyun motorları (Unity, Unreal), bu pseudo-mantığı çok daha hızlı ve optimize şekilde, arka planda milyonlarca kez çalıştırır. Bu simülasyonun amacı, sizden iki bilgisayar arasında mesaj göndermenizi değil; *hangi verinin, ne zaman, neden ve kime gönderileceğinin* mantığını kavramanızı sağlamaktır.

Özet Çıkarım: İstemci, "Ne yapmak istediğini" (Input) gönderir; Sunucu, "O dünyada gerçekte ne olduğunu" (State) hesaplar ve tüm istemcilere "Nasıl görünmesi gerektiğini" (Update) bildirir.

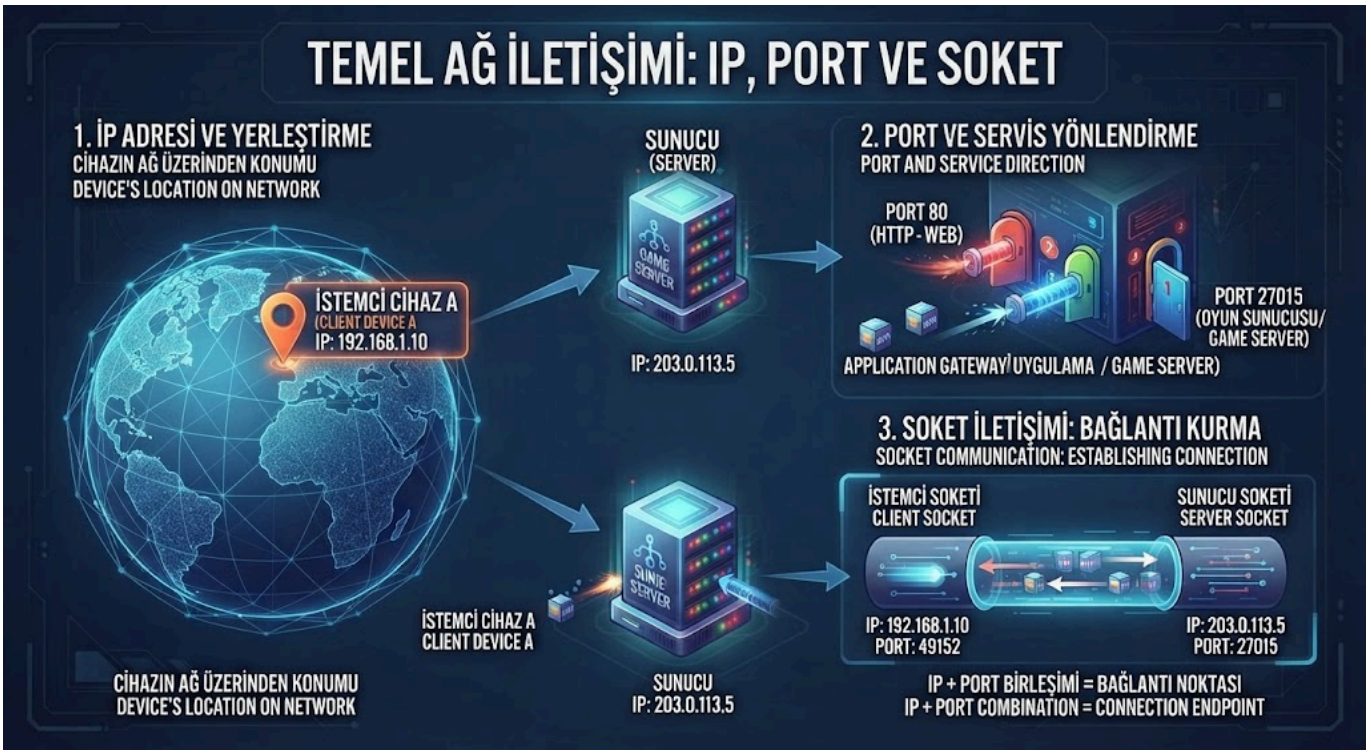
**Networking Temelleri - 2. **

Bu bölümde, çevrimiçi oyunların temelini oluşturan ağ iletişiminin yapı taşlarını inceleyeceğiz. Bir oyunun internetteki başka bir bilgisayarla nasıl konuştuğunu anlamak için IP, Port, Socket kavramlarını ve veri iletiminde kullanılan iki temel protokol olan TCP ile UDP arasındaki farkları detaylı olarak ele alacağız.

---

1. IP, Port ve Socket Kavramları

Bir oyun istemcisinin (sizin bilgisayarınız) bir oyun sunucusuna mesaj gönderebilmesi için, gönderilen verinin doğru adrese ulaşması gerekir. Bu adresleme mekanizması üç temel kavram üzerine kuruludur:



1.1. IP Adresi (Internet Protocol Address)

IP adresi, bir ağa bağlı her cihaza (bilgisayar, telefon, oyun konsolu, sunucu) atanan benzersiz sayısal etikettir. İnternet üzerindeki posta adresi gibi düşünülebilir.

- **İşlevi:** Veri paketlerinin doğru binaya (cihaza) ulaşmasını sağlar.
- **Türleri:**
  - **IPv4:** 32 bitlik adreslerdir (örnek: `192.168.1.1`). Günümüzde en yaygın kullanılan formattır, ancak adres sayısı sınırlıdır. (2^{32})
  - **IPv6:** 128 bitlik adreslerdir (örnek: `2001:0db8:85a3:0000:0000:8a2e:0370:7334`). Daha fazla adres imkanı sunar ve geleceğin standardıdır.
- **Oyunlardaki Önemi:** Bir oyun sunucusuna bağlanmak istediğinizde, DNS (Domain Name System) sunucusu, yazdığınız `oyun.com` gibi bir alan adını, sunucunun IP adresine çevirir. Ping değeriniz, büyük ölçüde sizin IP adresinizle sunucunun IP adresi arasındaki fiziksel uzaklığa ve yönlendirici (router) sayısına bağlıdır.

1.2. Port Numarası (Port Number)

Bir IP adresi, binanın adresidir. Ancak bir binada (bilgisayarda) aynı anda birden fazla uygulama çalışır. Örneğin, bir tarafta oyununuz açıkken, diğer tarafta bir web tarayıcısı veya Discord çalışıyor olabilir. Gelen veri paketinin hangi uygulamaya gitmesi gerektiğini belirlemek için port numarası kullanılır.

- **İşlevi:** IP adresine gelen veriyi, doğru uygulamaya (veya doğru oyun oturumuna) yönlendirir. Bir apartman dairesindeki daire numarası gibi düşünülebilir.
- **Aralığı:** 0 ile 65535 arasında değerler alır.
  - **0-1023:** Sistem portları (well-known ports). Örn: HTTP için 80, HTTPS için 443.
  - **1024-49151:** Kayıtlı portlar (registered ports). Örn: Çoğu oyun sunucusu bu aralıkta çalışır (Valve oyunları genelde 27015, Minecraft 25565).
  - **49152-65535:** Dinamik/Özel portlar (dynamic/private ports). İstemci uygulamalar tarafından geçici olarak kullanılır.
- **Oyunlardaki Önemi:** Bir oyun sunucusu kurarken, hangi portu dinleyeceğini (listen) belirtmeniz gerekir. İstemci oyuncular, bağlanmak için hem sunucunun IP adresini hem de bu port numarasını bilmelidir (örnek: `192.168.1.100:27015`).

1.3. Soket (Socket)

Soket, bir IP adresi ve bir port numarasının birleşiminden oluşan, iki uç nokta (endpoint) arasındaki iletişim kapısıdır.

- **Tanımı:** Soket = IP Adresi + Port Numarası. Veri göndermek veya almak için kullanılan programlama arayüzüdür (API).
- **İşlevi:** Bir istemci-sunucu iletişimini tanımlar. Örneğin, sunucu `213.74.23.100:8080` adresinde bir soket oluşturur ve bağlantıları bekler. İstemci, bu sokete bağlanarak veri alışverişine başlar.
- **Oyunlardaki Önemi:** Oyun motorları, arka planda soketler oluşturur. Her bağlantı, benzersiz bir soket çifti (Client Soket + Server Soket) ile temsil edilir. Bu sayede sunucu, binlerce bağlantıyı aynı anda yönetebilir.

---

2. TCP vs UDP Farkları (Oyun Perspektifi)

İki cihaz arasında veri göndermek için iki ana protokol kullanılır: TCP (Transmission Control Protocol) ve UDP (User Datagram Protocol). Bu iki protokol, oyun geliştirmede temel bir ayrım noktasıdır.

Özellik	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Bağlantı Türü	Bağlantı tabanlıdır (Connection-Oriented). Veri göndermeden önce 3 aşamalı bir el sıkışma (3-way handshake) ile bağlantı kurar.	Bağlantısızdır (Connectionless). Veriyi doğrudan gönderir, önceden bağlantı kurmaz.
Güvenilirlik (Reliability)	**Güvenilirdir.** Gönderilen her paketin karşı tarafa ulaşıp ulaşmadığını kontrol eder. Kayıp paketleri tekrar gönderir (Retransmission).	**Güvenilir değildir.** Paketlerin ulaşıp ulaşmadığını kontrol etmez. Kaybolan paketler için yeniden gönderme yapmaz.
Sıralama (Ordering)	Paketlerin sırasını garanti eder. Eğer paketler sırasız gelirse, doğru sıraya koyar.	Paketlerin sırasını garanti etmez. Sırasız gelebilirler.
Akış Kontrolü (Flow Control)	Veri akışını yönetir, alıcının kaldırabileceğinden fazla veri göndermesini engeller.	Akış kontrolü yoktur. Veriyi ne hızda göndereceği uygulamaya bağlıdır.
Hız (Speed)	Daha **yavaştır.** Güvenlik kontrolleri, paket sıralaması ve yeniden gönderme mekanizmaları ekstra zaman ve bant genişliği gerektirir.	Daha **hızlıdır.** Herhangi bir kontrol mekanizması olmadığı için "ateş et ve unut" (fire and forget) mantığıyla çalışır.
Kullanım Alanı (Oyun)	Kritik olmayan, sıralamanın ve ulaşmanın önemli olduğu veriler için.	Hızın kritik olduğu, paket kaybının tolere edilebildiği anlık veriler için.

TCP'nin "Hoş Geldin" Konuşması (3-Way Handshake):

1. İstemci: "Merhaba, bağlanmak istiyorum." (SYN)
2. Sunucu: "Tamam, hazırım." (SYN-ACK)
3. İstemci: "Anlaştık!" (ACK)

---

3. UDP vs TCP Kullanım Senaryoları Örnekleri

Hangi protokolün kullanılacağına oyunun türü ve gönderilecek verinin önemi karar verir. Günümüz oyunlarının çoğu, bu iki protokolü birlikte (hibrit) kullanır.

3.1. UDP'nin Kullanıldığı Senaryolar (Hız > Güvenilirlik)

- **Oyuncu Hareket Verileri (Position Updates):** Bir oyuncunun anlık konum ve yön bilgileri saniyede 10-60 kez gönderilir. Bu paketlerden birkaçı kaybolsa bile, bir sonraki gelen paketle durum güncellenir. Kaybolan eski bir hareket paketini yeniden göndermenin hiçbir anlamı yoktur, çünkü o oyuncu çoktan başka bir konuma gitmiştir. **UDP tercih edilir.**
- **Ateş Etme ve Mermi Verileri (Shooting / Projectiles):** Rekabetçi FPS oyunlarında (Valorant, CS:GO), atış anında gönderilen verilerin hızlı ulaşması hayati önem taşır. Arada bir paket kaybı yaşanırsa, "ben ateş ettim" hissi oyuncuda oluşmaz. Küçük kayıplar kabul edilebilir. **UDP tercih edilir.**
- **Sesli / Görüntülü İletişim (VoIP):** Oyun içi sesli sohbetlerde, birkaç milisaniyelik ses parçasının kaybolması, konuşmayı anlaşılmaz kılmaktan ziyade minik bir çatırtıya neden olur. Kayıp paketi

tekrar göndermek, sesin gecikmeli gelmesine ve konuşmanın kopmasına yol açar. **UDP tercih edilir.**

3.2. TCP'nin Kullanıldığı Senaryolar (Güvenilirlik > Hız)

- **Oyun Girişi ve Kimlik Doğrulama (Login / Authentication):** Kullanıcı adı ve şifre bilgilerinin karşı tarafa eksiksiz ve hatasız ulaşması gerekir. Paket kaybı yaşanırsa şifre yanlış algılanabilir veya oturum açılamaz. **TCP tercih edilir.**
- **Karakter Envanteri ve Eşya Değişimleri (Inventory / Trading):** Bir oyuncunun envanterindeki eşyaların veya oyun içi satın alımların kaybolmaması veya çoğalmaması gerekir. Bu veritabanı işlemleridir ve sıralı, eksiksiz teslimat şarttır. **TCP tercih edilir.**
- **Oyunun Yama ve Güncelleme Dosyalarının İndirilmesi (Patches / Downloads):** Oyun dosyalarının (asset) doğru bir şekilde, bozulmadan inmesi gerekir. Hatalı bir bayt, oyunun çökmesine neden olabilir. **TCP tercih edilir (HTTP/HTTPS üzerinden).**

3.3. Hibrit Kullanım Örneği (Oyunlarda Genel Yaklaşım)

Modern 3A oyunlarda şöyle bir yaklaşım izlenir:

- **UDP Soketi:** Oyunun canlı oynanış verileri için (Konum, Ateş, Hasar, Animasyon tetikleyicileri) kullanılır. Buraya gelen veriler hızlı işlenir.
- **TCP Soketi (veya WebSocket):** Oyun içi sohbet mesajları, arkadaş listesi güncellemeleri, maç sonu istatistiklerin gönderilmesi gibi işlemler için kullanılır.

Özet: Hangi Oyun Hangi Protokolü Kullanır?

Oyun Türü	Tercih Edilen Protokol	Neden?
FPS (CS:GO, Valorant)	UDP (Oynanış) / TCP (Sunucu Bağlantısı)	Milisaniye hassasiyeti. Arada kaybolan bir atış verisi, eski veriyi beklemekten iyidir.
MOBA (LoL, Dota 2)	UDP (Oynanış) / TCP (Sohbet)	Anlık yetenek ve hareket verileri hızlı gitmeli. Sohbetin güvenli ulaşması gerekir.
MMORPG (WoW, FFXIV)	Hibrit (UDP ağırlıklı)	Dünyada dolaşma anlıktır (UDP), ancak envanter ve ticaret işlemleri sağlamdır (TCP).
Strateji (Age of Empires)	UDP (Genellikle)	Çok sayıda birimin anlık durum güncellemeleri hızlı iletilmelidir.
Dövüş (Street Fighter)	UDP (Neredeyse tümü)	Hata düzeltmesi yapmak yerine, bir sonraki frame'in gelmesi daha önemlidir.

Oyunlarda doğru protokolü seçmek, doğru aracı doğru iş için kullanmaya benzer. UDP, hız gerektiren, tolere edilebilir kayıpları olan oyun mantığı için tercih edilirken; TCP, veri bütünlüğünün ve sıralamanın mutlak öncelik olduğu sistemik işlemler için vazgeçilmezdir. Bir oyun geliştirirken, her veri türü için bu maliyet-fayda analizini yapmak, oyunun akıcılığını ve kararlılığını doğrudan belirler.